

AgentFold

Long-Horizon Web Agents with Proactive Context Management - 深度研究与工程 Know-how

从“历史摘要”到“多尺度状态折叠”：让 Agent 主动决定什么该保留、什么该抽象、什么该忘记

长程 Agent Context Management：从日志累积到主动状态塑形

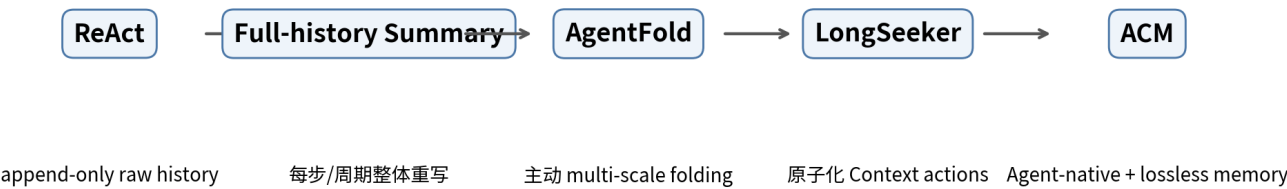


图1 Context Management 演进中的 AgentFold

研究日期：2026-08-13 | 主来源：AgentFold arXiv:2510.24699 + Tongyi DeepResearch family

目录与执行摘要

1. 一页结论：AgentFold 到底创新在哪里
2. Know-why：ReAct 与全历史摘要的双重失败
3. Cognitive Workspace：Multi-Scale Summaries + Latest Interaction
4. Folding Directive：Granular Condensation vs Deep Consolidation
5. Perceive -> Reason -> Fold -> Act 的完整状态转移
6. Fold-Generator：为什么只靠 Prompt 不够
7. 训练设计：SFT 是 Proof-of-Concept，而非终点
8. 实验结果：四项 benchmark 与模型规模效率
9. Context Dynamics：100 turns / 500 turns
10. Case Study：如何把 11-step dead end 折叠成一个结论
11. 与 ReSum / LongSeeker / AdaCoM / ACM 的关系
12. Know-how：如何在生产级 Deep Research 里复用
13. 局限与 2026 后视角
14. MVP 实现蓝图
15. 结论与资料来源

执行摘要：AgentFold 的核心贡献，是把“context summarization”改造成“agent-controlled multi-scale state transition”。过去历史不再是单一长日志，也不是每一步都被整体重写，而是由若干覆盖连续 step ranges 的 summary blocks 组成；最新一次 interaction 则保留完整高保真细节。每一步 Agent 同时决定折叠范围与下一次工具动作，使 context curation 与 task policy 紧密耦合。

1. 一页结论：AgentFold 到底创新在哪里

AgentFold 解决的是长程 Web Agent 的一个核心矛盾：ReAct 保留全部原始历史，信息完整但噪声和成本持续膨胀；每步/周期全历史摘要能降低长度，却反复重写同一批旧信息，任何一次摘要丢失关键细节都会不可逆累积。

- 状态表示：Question + Tools + Multi-Scale State Summaries + Latest Interaction。
- 每一步输出：Thinking + Folding Directive + Explanation + Tool Call / Final Answer。
- Granular Condensation：只把最近一步压成细粒度 summary；用于保留尚未确认价值的关键细节。
- Deep Consolidation：把最近一步与若干更早 summary blocks 一起折叠成更粗粒度 conclusion；适合已完成子任务或确认死路。
- 训练：Fold-Generator 生成结构化轨迹，经过 rejection sampling 后，对 Qwen3-30B-A3B-Instruct-2507 做 SFT。论文明确没有使用 continual pre-training 或 RL。
- 结果：BrowseComp 36.2、BrowseComp-ZH 47.3、WideSearch 62.1、GAIA 67.0。
- Context 证据：约 100 turns 时平均 context 只有 7K tokens；相同轨迹的 ReAct 约多 84K tokens，论文给出约 92% reduction。

AgentFold 的 Cognitive Workspace

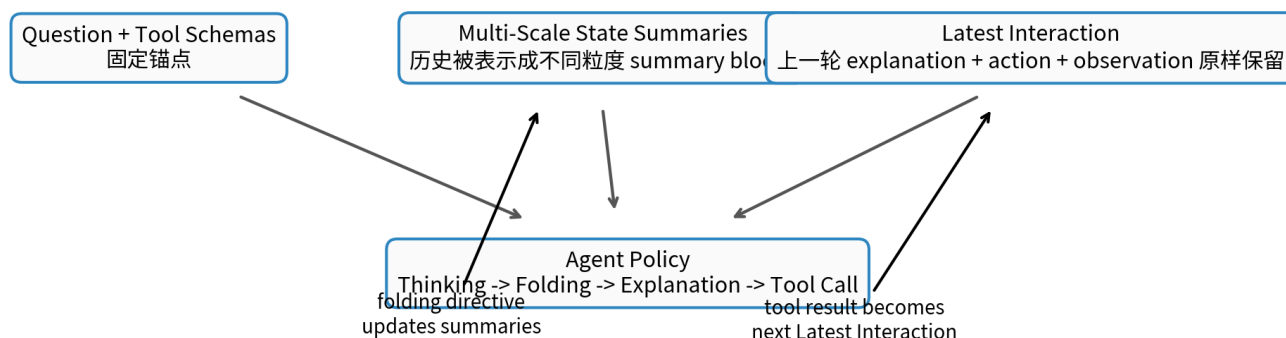


图2 AgentFold 的动态 Cognitive Workspace

Source: AgentFold arXiv HTML, Sec. 3.1-3.3, Fig. 2; 2025-10-28 v1.

2. Know-why：ReAct 与全历史摘要的双重失败

2.1 ReAct 的问题不是只有“超过窗口”

Append-only ReAct 会把 reasoning、action、observation 持续堆入下一步 prompt。即使硬 context window 尚未耗尽，模型仍要在大量失败网页、重复证据和过时假设中重新定位当前任务状态。问题因此同时包含 token cost、attention dilution、重复检索和错误分支污染。

2.2 全历史摘要的隐含风险：重复重写

如果每一步都把整段历史重新生成一次 summary，那么一个很早发现的关键事实会经历许多次“摘要的摘要”。论文用一个假设例子说明 compounding loss：若每轮重写只有 1% 的关键细节丢失概率，step 1 的信息到 step 100 仍存活的概率约为 $0.99^{100} = 36.6\%$ ；到 step 500 约只剩 0.66%。这不是实验统计，而是用来解释反复全量重写为什么在极长 horizon 下结构性危险。

2.3 AgentFold 的关键转变：retrospective consolidation

AgentFold 不强迫旧信息每轮重新摘要。它允许某个细粒度 summary block 长时间保持不变，直到后续证据足够明确时才把若干 block 一次性合并成更高层 conclusion。也就是说，压缩时机由任务语义成熟度决定，而不是由 clock / token threshold 决定。

Source: AgentFold Sec. 3.5, including the illustrative 1% loss calculation.

3. Cognitive Workspace：Multi-Scale Summaries + Latest Interaction

论文形式化为 $C_t = (Q, T, S_{t-2}, I_{t-1})$ 。其中 Q 是问题、T 是工具集合、S 是 Multi-Scale State Summaries、I 是最近一次完整 interaction。

组件	信息粒度	作用	工程对应
Question	固定	持续锚定最终目标	task contract
Tool Schemas	固定/慢变	定义可执行动作空间	tool registry
Multi-Scale State Summaries	多粒度	长期、结构化任务状态	working memory index
Latest Interaction	高保真原始细节	短期 situational awareness	hot buffer

S_t 不是一段自由文本，而是有顺序的 summary blocks: $s_{x,y}$ 表示对连续 steps $x..y$ 的摘要。所有 block 的 step range 连续覆盖过去历史，因此 Agent 既知道“发生了什么”，也知道每个 summary 对应哪一段时间范围。

- Fine block $s_{x,x}$: 一个 step 一个摘要，适合仍可能需要细节的发现。
- Coarse block $s_{x,y}$: 多个 steps 一个摘要，适合已形成稳定结论的子过程。

这个表示的 know-how 是“显式保留 temporal provenance”。即使真正文本被压缩，summary block 仍携带原始 step range，使 runtime 能判断覆盖关系、避免重叠、实现 deterministic replacement。

4. Folding Directive : Granular Condensation vs Deep Consolidation

一个 folding JSON 格式，支持两种不同时间尺度



核心：不是“定期总结”，而是推理状态驱动的 retrospective consolidation

图3 两种 Folding 尺度

Folding directive 的统一格式可写成 `{"range": [k, t-1], "summary": sigma_t}`。Runtime 撤回落在 $[k, t-1]$ 内的旧 summary blocks，并以新的 $s_{[k, t-1]}$ 替换。

4.1 Granular Condensation

当 $k=t-1$ ，只压缩 Latest Interaction。它不是为了最大 compression ratio，而是为了把最近一次冗长 observation 变成“高分辨率可复用状态”。适合候选实体、尚待验证的假设、关键 URL、部分条件满足等中间发现。

4.2 Deep Consolidation

当 $k < t-1$ ，Agent 会把最近 interaction 与一串更早 summary blocks 融合成一个更抽象的 block。最典型触发条件不是 token 超限，而是“这个子任务已经完成/已经证明走不通”。

4.3 设计意义

- 同一个动作格式覆盖多尺度，不需要设计很多 meta tools。
- 压缩边界是连续 step range，Host Runtime 容易验证、应用和审计。
- 深度折叠可以让失败分支“保留教训但删除过程”，这本质上是 trajectory-level reflection。

Source: AgentFold Sec. 3.3, equations and folding directive format.

5. Perceive -> Reason -> Fold -> Act 的完整状态转移

AgentFold 最值得迁移的不是某个 summary prompt，而是状态转移顺序：先基于当前 workspace 推理，再决定如何改写历史，之后才执行新的外部 action。Context curation 与下一步搜索因此共享同一个意图。

阶段	输入	输出	为什么重要
Perceive	Q,T,S,I	当前可见状态	只把高价值 working set 给模型
Reason	current workspace	对任务与历史价值的联合判断	同时规划 context 与 task action
Fold	folding directive	S -> S'	把历史变成更适合下一步的表示
Act	tool call	observation	继续获取环境信息
Update	explanation/action/ observation	new Latest Interaction	最近一步继续保持高保真

一个 subtle but important 的点：folding directive 与 tool call 是同一轮模型输出的两个并行决策。它们不是“先压缩完再让模型重新思考一次”。这样可以减少额外 LLM round-trip，并让“我为什么要查这个”和“为了查这个，旧历史应该保留什么”形成联合策略。

6. Fold-Generator：为什么只靠 Prompt 不够

Fold-Generator：先 generate-and-filter，再用 SFT 蒸馏成单模型策略



论文刻意不使用 RL：目标是先证明 proactive folding paradigm 本身是否成立

图 4 Fold-Generator 数据构造流程

论文报告，即使很强的通用 LLM，通过 prompting 也难以稳定生成 AgentFold 要求的 structured multi-part response。问题并不只是 JSON 格式，而是一次输出同时要正确完成 thinking、fold range selection、summary synthesis、explanation 和 tool call。

因此作者建立 Fold-Generator：用强开源模型生成候选交互，再使用 rejection sampling 丢弃两类失败样本：一是结构/格式不满足协议的 step；二是含有过多环境错误的 trajectory。最终得到 (C_t, R*_t) interaction pairs。

- Know-how：复杂 Agent protocol 如果只靠 system prompt 维持，长期运行中格式错误会快速累积；应将“协议遵守”直接训练进模型。
- Know-how：训练样本粒度可以是 step-level interaction pair，而不必把整个长 trajectory 一次塞入 SFT。
- Know-how：rejection sampling 不仅过滤答案质量，还应过滤 runtime/tool errors，否则模型会学到无效环境行为。

Source: AgentFold Sec. 3.4.

7. 训练设计：SFT 是 Proof-of-Concept，而非终点

最终模型基于 Qwen3-30B-A3B-Instruct-2507：总参数约 30B、推理激活约 3B。实验最大工具调用数设为 100，超过则强制终止。

论文刻意强调：AgentFold 没有 continual pre-training，也没有 RL。这个设计让实验更容易回答一个干净问题：只把“folding policy”通过监督轨迹内化，是否已经能显著改善 long-horizon search？结果显示答案是肯定的。

7.1 但 SFT 的结构性限制

- Teacher 的 folding 偏好会被直接模仿，Student 很难发现 teacher 没示范过的更优折叠边界。
- summary quality、tool success 与最终 answer quality 没有通过同一个 end-to-end reward 直接优化。
- compression/cost trade-off 没有显式价格信号：模型不知道少 1K token 到底值多少。

作者在结论中也明确把 RL 作为下一步：让 Agent 直接以 task success 学习潜在的非显然 folding policy。后续 2026 的 LongSeeker、AdaCoM、ACM 分别沿着更细 Context action、外置 Context RL、lossless memory 等方向继续扩展。

8. 实验结果：四项 benchmark 与模型规模效率

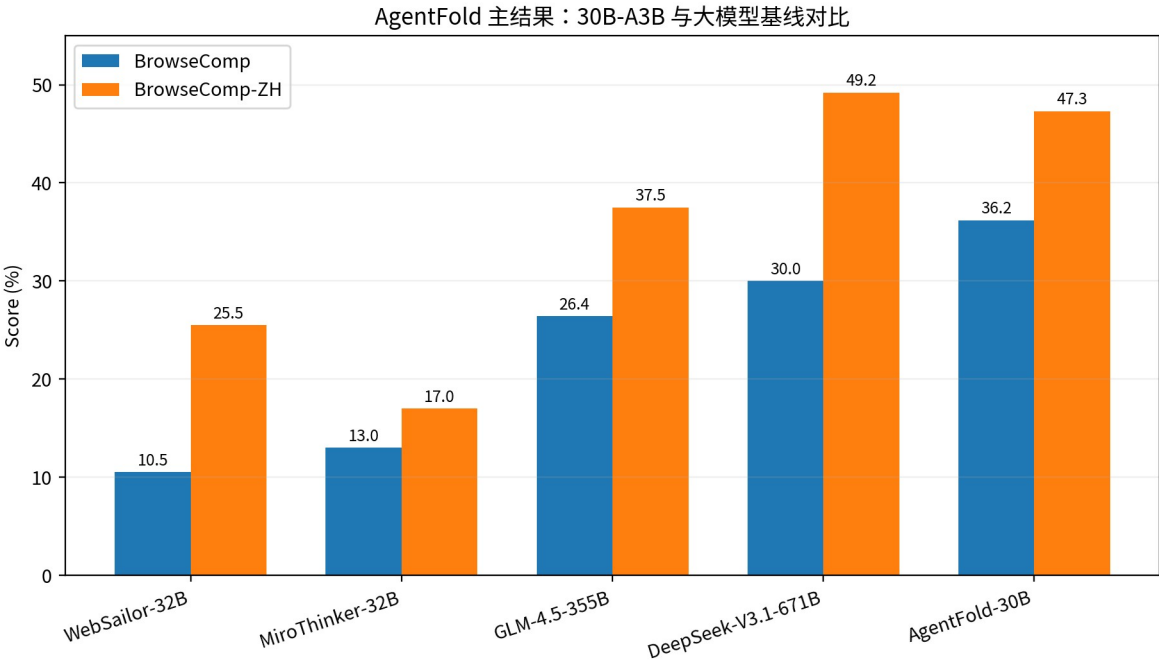


图5 BrowseComp / BrowseComp-ZH 代表性对比 (论文 Table 1)

Benchmark	AgentFold	测什么	备注
BrowseComp	36.2	hard-to-find web information	论文主长程检索指标
BrowseComp-ZH	47.3	中文浏览与多跳定位	略低于 DeepSeek-V3.1 49.2
WideSearch	62.1	broad information seeking / Item-F1	论文表中略高于 Claude-4-Sonnet 62.0 与 OpenAI-o3 60.0
GAIA	67.0	general agent capability	接近 proprietary baselines

最值得注意的不是简单“30B 打败 671B”。不同系统的工具、预算、训练数据与评测设置可能不同，不能把 Table 1 当作纯模型能力对比。更可靠的结论是：在作者采用的 benchmark setup 中，主动 context folding 的 30B-A3B Agent 具备很强的长程搜索竞争力。

Source: AgentFold Table 1, Sec. 4.1. Baseline numbers partly taken from corresponding papers/leaderboards, per authors.

9. Context Dynamics : 100 turns / 500 turns

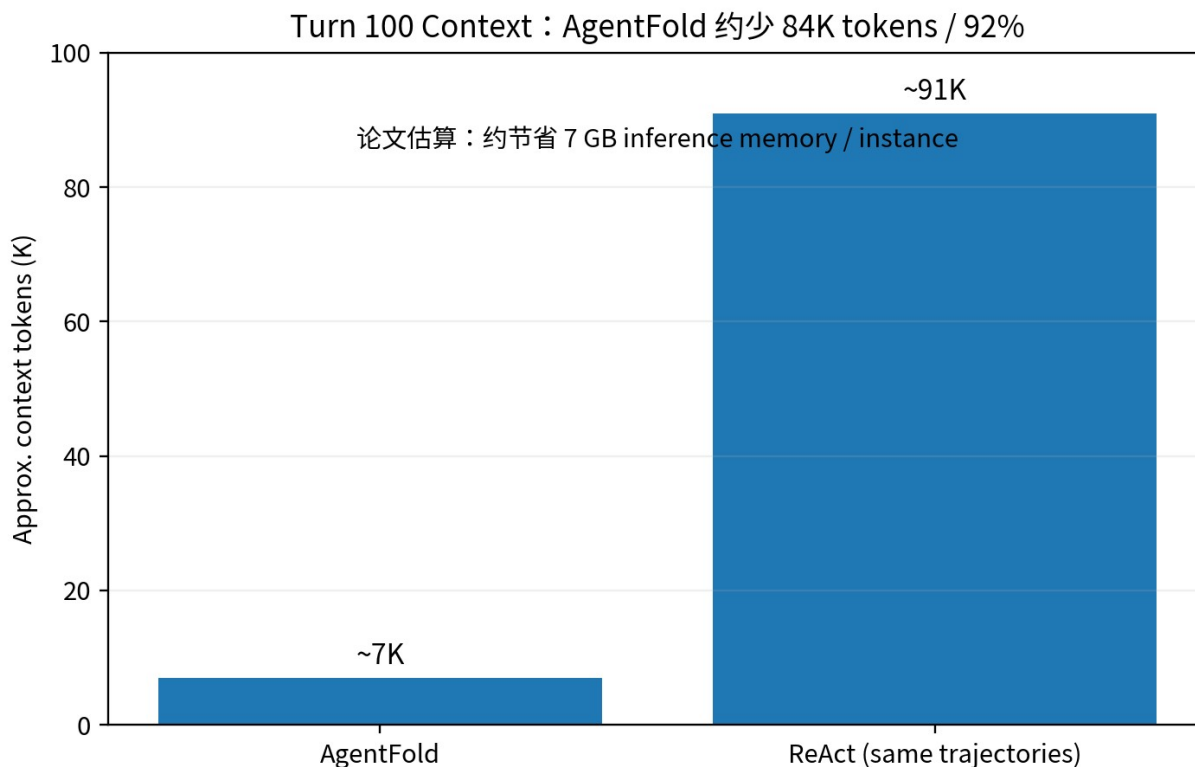


图 6 Turn 100 的 Context Efficiency

作者分析 BrowseComp 的 200 条 trajectory: AgentFold 平均 context token count 在 100 turns 内从约 3.5K 增长到约 7K, 呈慢速、次线性增长。实验模型有 128K context, 因此 100 turns 结束时并非被模型窗口卡死。

- 超过 20% 的任务在 100-turn 人工上限处被强制终止, 通常记作失败; 这意味着 interaction budget 而不是 context window 成为新的瓶颈。
- 与相同 trajectory 的 ReAct 对比, 100 turns 时 AgentFold 平均少约 84K tokens, 即约 92%。论文估算这相当于每个 inference instance 接近 7GB memory saving。
- 将最大 turns 扩到 256 时, AgentFold accuracy 继续提升; GLM-4.5 append-only agent 在 64 turns 之后出现饱和/无法继续。

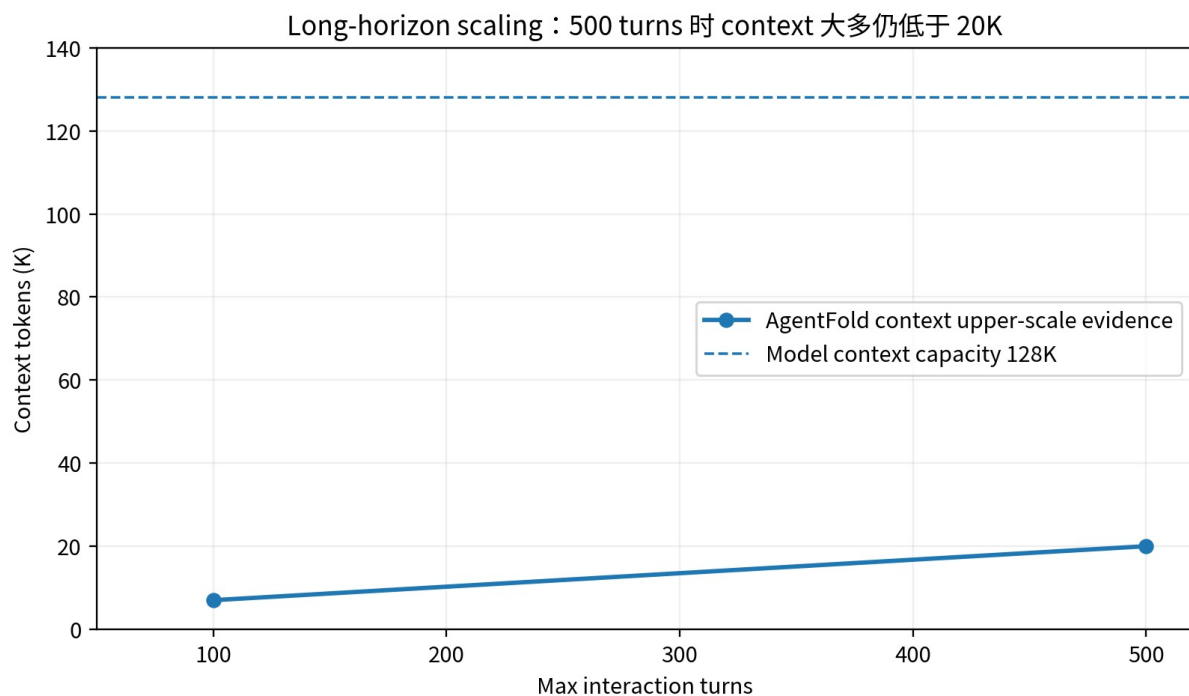


图7 极长 horizon 的 Context Scaling 证据

500-turn 扩展实验中，AgentFold context 大多仍低于 20K，而且不会单调增长。当一条长探索路径被判定为 dead end，Deep Consolidation 会把整段失败历史折叠成一个短结论，使 context 重新变紧。

Source: AgentFold Fig. 1, Fig. 3, Fig. 4 and Sec. 4.1.

10. Case Study：如何把 11-step dead end 折叠成一个结论

论文 case study 的关键节点发生在 step 17。此前 context 同时包含若干 fine-grained 单步 summaries，以及 [Compressed Step 6 to 8] 这样的较粗 block。随后 Agent 识别 step 6..16 是一整段失败方向。它执行 Deep Consolidation，把这 11 个 steps 替换成一个“这条路线不可行”的结论，然后立即切换新的 search direction。这里真正有价值的是：失败本身没有被完全忘记，保留下来的是 failure lesson；被删除的是不再需要重复阅读的 procedural noise。

传统处理	AgentFold 处理	效果
全量保留失败 11 steps	折叠成一个 failure conclusion	减少噪声与 token
完全删除失败路径	保留“为什么失败”	避免再次走回相同死路
每轮都重写所有历史	只重写需要变粗的连续区间	降低重复摘要造成的信息漂移

可迁移 Know-how：失败轨迹应该被“语义压缩”为 anti-pattern / excluded hypothesis，而不是简单删除。对 Deep Research 来说，这可以表示为 rejected candidate、contradicted claim、blocked source、failed query family。

Source: AgentFold Fig. 5 and Appendix A case study.

11. 与 ReSum / LongSeeker / AdaCoM / ACM 的关系

Context Management Design Space : AgentFold 是“主动多尺度折叠”的早期关键节点

方法	触发者	操作粒度	历史可恢复性	学习方式
ReSum	外部阈值/周期	整体 summary restart	不可恢复	summary model + ReSum-GRPO
AgentFold	主 Agent 每步	多尺度 fold range	不可恢复	SFT
LongSeeker	主 Agent 每步	Skip/Compress/Rollback/Snippet/Delete 操作区可编辑，不内建冷存储		SFT
AdaCoM	外部 manager	rewrite/merge/delete/no-change	取决于系统	Context Manager RL
ACM	主 Agent 主动	manage_context + query_memory	可恢复 / lossless	OPD/KD post-training

图 8 2025-2026 Context Management 设计空间

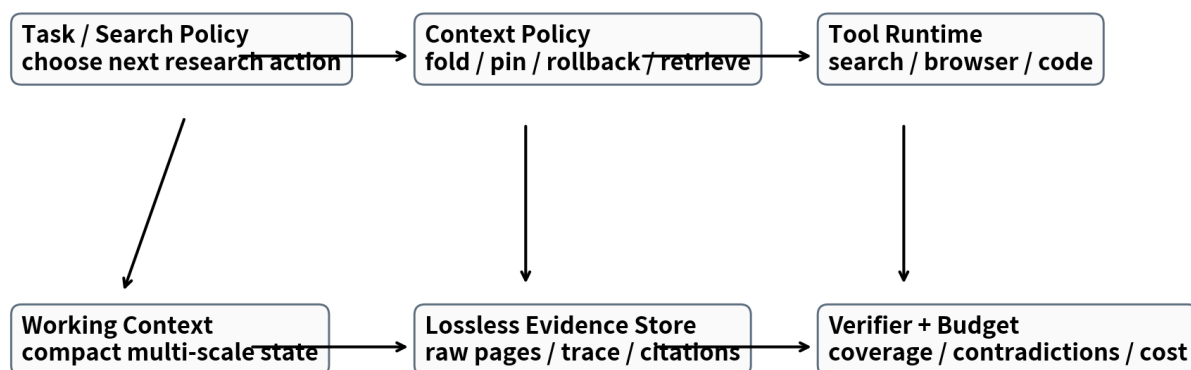
AgentFold 是一个非常重要的中间节点：它第一次把“何时折叠、折叠多少”深度耦合进 Agent action policy，但它的 action space 仍然主要是一个 range + summary。后续方法把这个思想继续拆开。

- ReSum：外部/周期 summary restart，改动小、插件化强，但压缩粒度较粗。
- AgentFold：主 Agent 主动、多尺度、range-based；适合让一个模型联合学习 task state 与 context state。
- LongSeeker / Context-ReAct：把单一 fold 扩成 Skip、Compress、Rollback、Snippet、Delete，更细地控制信息保真度。2026 论文在 BrowseComp / ZH 上分别报告 61.5 / 62.5，明确高于 AgentFold 的 36.2 / 47.3。
- AdaCoM：冻结主 Agent，训练独立 Context Manager RL，适合不能改主模型或希望跨 Agent 迁移。
- ACM：重新强调 lossless，把压掉的原始 context offload 到长期 memory，再通过 query_memory 召回。

所以从 2026 后视角看，AgentFold 的历史价值不是“当前最强”，而是它把 Context Management 从 passive compression 推进成了 agent-native decision。

Source: LongSeeker arXiv:2605.05191; AdaCoM arXiv:2605.30785; ACM arXiv:2607.23809.

12. Know-how：如何在生产级 Deep Research 里复用



生产原则：Working Context 可以有损；原始 Evidence / Trace 不应有损

图9 生产级升级：Context Policy 与 Lossless Evidence Store 分离

12.1 Multi-scale State 不要只存 prose summary

生产系统应把一个 folded block 设计成结构化对象，而不是只有 summary string，例如：range、summary、claims、evidence_ids、rejected_candidates、open_questions、confidence、created_at、source_domains。这样下一轮可以检索和验证，而不需要仅凭自然语言记忆。

12.2 Folding 不应破坏 provenance

AgentFold 原论文的 summary 是有损的。生产升级应保留 raw trace / raw webpage / citation passage 在冷存储中，folded block 只作为 working view。这样一旦 summary 被怀疑，可以回到原始 evidence。

12.3 触发条件从“token budget”升级为“semantic maturity”

- subtask completed -> deep consolidate
- dead end confirmed -> fold into failure lesson + pivot
- precision-critical evidence -> pin/snippet, 不做生成式重写
- contradiction unresolved -> 暂缓 consolidation, 保留多视角原文

12.4 Host Runtime 必须 deterministic

模型负责提出 range 与 summary；Host 负责验证 range 连续性、禁止越界、处理 summary block 覆盖关系、写 trace store、计算 token/cost。不要让模型自由修改任意历史文本。

13. 局限与 2026 后视角

局限	为什么重要	2026 可行升级
摘要有损且不可召回	早期错误折叠可能永久损失证据	ACM-style lossless offload + retrieve
只用 SFT	模仿 teacher，不直接优化 task success/cost	RL / preference optimization for folding policy
range + summary action 较粗	不能精确 pin/delete/rollback	LongSeeker-style atomic context actions
没有显式 claim/evidence graph	summary 可能混合多个来源与置信度	claim-level provenance + contradiction edges
100-turn 主实验受人为预算限制	性能瓶颈从 context 转为 action budget	adaptive stop / test-time scaling
表中跨系统 baseline 设置不统一	不能当作纯模型 size 因果证据	统一工具、预算、judge 的 controlled eval

另一个重要后续是 ICML 2026 的 “Scaling Long-Horizon LLM Agent via Context-Folding”（FoldAgent）。它与 AgentFold 名称相近但不是同一工作：该方法让 Agent branch into sub-trajectory，完成子任务后再 fold，并用 FoldGRPO 强化学习 context folding。两者共同说明 2025-2026 的趋势是：Context Management 正从 heuristic compression 变成可训练的 agent capability。

Source: AgentFold conclusion; ICML 2026 OpenReview “Scaling Long-Horizon Agent via Context Folding” .

14. MVP 实现蓝图

如果要在现有 Deep Research Agent 中最小化实现 AgentFold 思想，不需要先训练模型。可以先把 runtime 状态结构化，再让通用模型通过 typed tool / constrained JSON 输出 folding directive。

```
State:
  question
  tools
  summaries = [{start_step, end_step, summary, evidence_ids}]
  latest_interaction = {explanation, action, observation}
  raw_trace_store = append_only

Loop:
  response = policy(state_view)
  validate(response.fold.range)
  new_summary = response.fold.summary
  summaries = replace_covered_blocks(summaries, range, new_summary)
  observation = execute(response.action)
  raw_trace_store.append(full_step)
  latest_interaction = full_step
  state_view = question + tools + summaries + latest_interaction
```

推荐分阶段

- Stage 1: Host 端 Multi-Scale Summary Blocks + deterministic range replacement。
- Stage 2: 只允许 granular fold；先验证格式稳定与 answer quality 不掉。
- Stage 3: 开放 deep consolidation，并记录 dead-end / completed-subtask 标签。
- Stage 4: 加入 raw evidence store + retrieve，避免不可逆信息损失。
- Stage 5: 构建 trajectory eval：正确率、context tokens、重复工具率、fold error、evidence recall、cost。
- Stage 6: 从 SFT 进入 RL / preference training，让 quality > evidence integrity > cost 成为可优化目标。

15. 结论与资料来源

AgentFold 的核心价值可以压成一句话：它把长程 Agent 的历史从“被动日志”变成“主动维护的多尺度状态”。Granular Condensation 保留局部细节，Deep Consolidation 把完成的子任务和失败分支提升为更高层结论；Latest Interaction 保留最近一步的高保真观察，使下一步决策不会因为过早压缩而失去局部线索。

从 2026 的演进看，更成熟的方向已经很清楚：AgentFold-style proactive folding + LongSeeker-style fine-grained actions + AdaCoM-style learned context policy + ACM-style lossless external memory + evidence verifier。

Long-Horizon Research Agent $\approx$ Task Policy x Context Policy x Multi-Scale State x Lossless Evidence Memory x Verifier x Budget Controller

主要资料

- AgentFold: Long-Horizon Web Agents with Proactive Context Management. arXiv:2510.24699, v1, 2025-10-28. <https://arxiv.org/abs/2510.24699>
- Alibaba-NLP/DeepResearch official repository. Current family README lists AgentFold as item #12 in the Tongyi Deep Research agent family. <https://github.com/Alibaba-NLP/DeepResearch>
- LongSeeker: Elastic Context Orchestration for Long-Horizon Search Agents. arXiv:2605.05191, 2026.
- ReSum: Unlocking Long-Horizon Search Intelligence via Context Summarization. arXiv:2509.13313, 2025.
- AdaCoM: Adaptive Context Management for Long-Horizon Agents. arXiv:2605.30785, 2026.
- ACM: Agentic Context Management for Long Horizon Tasks. arXiv:2607.23809, 2026.
- Scaling Long-Horizon LLM Agent via Context-Folding. ICML 2026 / arXiv:2510.11967.

证据层级说明：AgentFold 架构、训练、主实验与 context dynamics 以原论文为一手证据；2026 的比较结论来自对应后续论文/官方仓库。对跨系统 leaderboard 数字仅作论文表内比较，不将其解释为严格的模型规模因果关系。